Preliminaries

Ricardo Andres Calvo Mendez

PUID: 36688820 · rcalvome@purdue.edu

ECE 69500 · Hardware Software Security

September 2, 2026

Santiago Torres Arias

---

1 Installing the Required Tools

I use Arch Linux as my operating system. I use `pacman` to manage packages.

I found that the package names listed in the assignment correspond directly to packages in `pacman`. I used the `--needed` flag to avoid reinstalling packages if they were already present in my system.

Terminal

```
→ ricardo ~ sudo pacman -S --needed arm-none-eabi-gcc
→ ricardo ~ sudo pacman -S --needed arm-none-eabi-binutils
→ ricardo ~ sudo pacman -S --needed qemu-system-arm
```

Now, I ran the `-Ql` flag on each of them to see which files those packages brought to my computer. I realized that the output was super long, so I redirected it to a file.

Terminal

```
→ ricardo ~ pacman -Ql arm-none-eabi-gcc > arm-none-eabi-gcc-files.txt
→ ricardo ~ pacman -Ql arm-none-eabi-binutils > arm-none-eabi-binutils-files.txt
→ ricardo ~ pacman -Ql qemu-system-arm > qemu-system-arm-files.txt
```

For `arm-none-eabi-gcc`: It contains 2,969 entries. Using the folders, I grouped the contents like this:

1. The path `/usr/arm-none-eabi/include/c++/` contains ∼1,650 entries (∼56%) consisting mainly of C++ headers.
2. The path `/usr/arm-none-eabi/lib/` contains ∼250 entries (∼9%) related to the required libraries for the package.
3. The path `/usr/bin/` contains ∼20 entries (∼1%) related to the executables. I assume this because it is included in my `$PATH`.
4. The path `/usr/lib/gcc/arm-none-eabi/` contains ∼1,000 entries (35%) related to GCC drivers.
5. The last path `/usr/share/man/` contains manual pages for the installed commands.

For `arm-none-eabi-binutils`: It contains 84 entries.

1. The path `/usr/arm-none-eabi/bin/` contains ∼10 entries (∼12%) consisting mainly of binaries.
2. The path `/usr/arm-none-eabi/lib/` contains ∼20 entries (∼24%) related to the required libraries for the package.
3. The path `/usr/bin/` contains ∼15 entries (∼18%) related to the executables.
4. The path `/usr/share/man/` contains the remaining files (∼46%) for manual pages.

For `qemu-system-arm`: It contains only 16 entries.

1. The executable in the path `/usr/bin/`.
2. Some licenses in the path `/usr/share/licenses/`.
3. Manual pages at the path `/usr/share/man/`.

2 qemu-system-arm

Now, for this part of the assignment, I tried to run the command provided to run the "hello world," but I got this message.

Terminal

```
→ ricardo homework1 qemu-arm cpu-emu-test
zsh: command not found: qemu-arm
```

Checking with my trusty LLM, I realized that I was missing the package `qemu-user`, so I installed it using `pacman` as I did before.

Terminal

```
→ ricardo homework1 sudo pacman -S --needed qemu-user
```

For `qemu-user`: It contains only 45 entries.

1. Some required tool executables in the path `/usr/bin/`.
2. Manual pages at the path `/usr/share/man/` as usual.

Now, it runs correctly!

Terminal

```
→ ricardo homework1 qemu-arm cpu-emu-test
Hello world!
```

Continuing with the assignment, I ran the command to emulate a machine:

Terminal

```
→ ricardo ~ qemu-system-arm -machine mcimx6ul-evk
VNC server running on ::1:5900
```

I was expecting a monitor interface as shown in the reading. I again asked my trusty LLM for options to solve this problem.

I have two options to address this:

1. To install `qemu-ui-gtk` that would provide my system with a GTK interface to the emulated machine.
2. To add the flags `-display none -monitor stdio` so it will directly open the monitor in my terminal emulator.

I chose option 2 because I prefer to avoid unnecessary packages. Now, my updated command works correctly:

Terminal

```
→ ricardo ~ qemu-system-arm -machine mcimx6ul-evk -display none -monitor stdio

QEMU 11.1.1 monitor - type 'help' for more information
(qemu)
```

Now the setup is completed!

3 Building ARM binaries

Let's save the file `babysteps.s` with the recommended formatting:

babysteps.s

```
1   .global _start
2
3   _start:
4   top:
5       eor r1, r1
6       b top
```

To build this, I ran the commands recommended in the readings:

Terminal

```
→ ricardo homework1 arm-none-eabi-as -g babysteps.s -o babysteps.o
→ ricardo homework1 arm-none-eabi-ld -g babysteps.o -o babysteps.elf
```

Investigating by myself, I was able to understand what those commands do:

- The first command `arm-none-eabi-as` builds an object file for ARM architecture. It reads the file `babysteps.s` and then the flag `-o` indicates the output path `babysteps.o`.
- The second command `arm-none-eabi-ld` links the object file assigning memory addresses and resolving symbols. It reads the file `babysteps.o` and then the flag `-o` indicates the output path `babysteps.elf`. The flag `-g` is just a flag for debugging tools.

Now, decomposing the assembly source code, I would describe it as follows:

- `.global _start` declares the symbol `_start` as global.
- `_start:` indicates that the symbol `_start` starts in that position.
- `top:` indicates that the symbol `top` starts in that position.
- `eor r1, r1` indicates an XOR operation. The evaluation of r1 $\oplus$ r1 is assigned to r1.
- `b top` indicates a jump to the `top:` symbol.

I would summarize the assembly as the following "equivalent" `C` code that is more human-readable:

babysteps.c

```c
int start(){
    int r1;
    while(1){
        r1 = r1 ^ r1;
    }
}
```

It contains an endless loop with an assignment of an XOR operation.

4 Studying the compiled binary

I ran the command `readelf` as shown in the reading.

Terminal

```
→ ricardo homework1 readelf -h babysteps.elf
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF32
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              EXEC (Executable file)
  Machine:                           ARM
  Version:                           0x1
```

```
Entry point address:               0x8000
Start of program headers:          52 (bytes into file)
Start of section headers:          4580 (bytes into file)
Flags:                             0x5000200, Version5 EABI, soft-float ABI
Size of this header:               52 (bytes)
Size of program headers:           32 (bytes)
Number of program headers:         1
Size of section headers:           40 (bytes)
Number of section headers:         8
Section header string table index: 7
```

After investigating documentation, the command `readelf` displays all the information related to ELF files. ELF files are usually executable files in Unix systems.

Summarizing the information there:

- **Magic:** Traditional file magic number. The bytes `7f 45 4c 46` correspond to the ELF magic number.
- **Class:** 32-bit ELF format.
- **Data:** Little-endian representation.
- **Version:** ELF format version.
- **OS/ABI:** Operating system and ABI for which the file was generated.
- **ABI Version:** Version of the specified ABI.
- **Type:** ELF file type. `EXEC` means that it is an executable file.
- **Machine:** Target processor architecture.
- **Version:** ELF object file version.
- **Entry point address:** Address where program execution begins.
- **Start of program headers:** Offset from the beginning of the file to the program header table.
- **Start of section headers:** Offset from the beginning of the file to the section header table.
- **Flags:** Architecture-specific flags.
- **Size of this header:** Size of the ELF header.
- **Size of program headers:** Size of each entry in the program header table.
- **Number of program headers:** Number of entries in the program header table.
- **Size of section headers:** Size of each entry in the section header table.
- **Number of section headers:** Number of entries in the section header table.

- **Section header string table index:** Specifies which section header contains the string table used to store section names.

I ran the command `readelf` with the flag `--section-headers` to get more information.

Terminal

```
→ ricardo homework1 readelf --section-headers babysteps.elf
There are 8 section headers, starting at offset 0x11e4:

Section Headers:
  [Nr] Name              Type            Addr     Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            00000000 000000 000000 00      0   0  0
  [ 1] .text             PROGBITS        00008000 001000 000008 00  AX  0   0  4
  [ 2] .persistent       PROGBITS        00009008 001008 000000 00  WA  0   0  4
  [ 3] .noinit           NOBITS          00009008 000000 000000 00  WA  0   0  4
  [ 4] .ARM.attributes   ARM_ATTRIBUTES  00000000 001008 000012 00      0   0  1
  [ 5] .symtab           SYMTAB          00000000 00101c 000120 10      6   8  4
  [ 6] .strtab           STRTAB          00000000 00113c 000062 00      0   0  1
  [ 7] .shstrtab         STRTAB          00000000 00119e 000045 00      0   0  1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
  L (link order), O (extra OS processing required), G (group), T (TLS),
  C (compressed), x (unknown), o (OS specific), E (exclude),
  D (mbind), y (purecode), p (processor specific)
```

Again, I investigated about the information here and I summarize it:

- **Index 0:** Empty by convention.
- **.text:** Executable instructions of the program. It is allocated in memory and executable.
- **.persistent:** Data intended to persist across certain executions or resets. It is writable and allocated.
- **.noinit:** Uninitialized data that requires memory space but does not need to be stored in the ELF file. It is writable and allocated.
- **.ARM.attributes:** ARM-specific metadata describing architecture and ABI attributes.
- **.symtab:** The symbol table contains information about functions and variables.
- **.strtab:** String table containing the names of functions and variables.
- **.shstrtab:** The string table with the names of the sections.

5 Disassembling the compiled binary

Let's disassemble the ELF file we got.

Terminal

```
→ ricardo homework1 arm-none-eabi-objdump -D babysteps.elf

babysteps.elf:     file format elf32-littlearm


Disassembly of section .text:

00008000 <_start>:
    8000:       e0211001        eor     r1, r1, r1
    8004:       eafffffd        b       8000 <_start>

Disassembly of section .ARM.attributes:

00000000 <.ARM.attributes>:
   0:   00001141        andeq   r1, r0, r1, asr #2
   4:   61656100        cmnvs   r5, r0, lsl #2
   8:   01006962        tsteq   r0, r2, ror #18
   c:   00000007        andeq   r0, r0, r7
  10:   Address 0x10 is out of bounds.
```

According to the output we got, I see in the `.text` section the code disassembled. The output includes memory addresses, but its logic is the same as the source code we had before.

Cleaning up the output of the disassembling command, I would recover the data like this:

babysteps_recovered.s

```
1   _start:
2       eor     r1, r1, r1
3       b       _start
```

I noticed the following points and I tried to give the best explanation I could:

1. `.global` **keyword is gone:** Investigating a little bit about this, the way to discover if a label is global is checking the symbol table. Using the command `readelf -s babysteps.elf`, I was able to see it as global.
2. **Label** `top:` **is gone:** In my experience working in compilers, it should've been removed by some sort of compiler optimization. It is clear that this label was not necessary due to the existence of top in the same memory address.
3. `eor` **instruction now has 3 parameters:** This is because the original `eor r1, r1` is just a shortcut of the 3-parameter version because the destination register is the same as the first operand of the XOR operation.

Essentially, the changes are for optimization, but the semantics of the sources are equivalent.

6 Running ARM binaries we just built

To run the binaries in an emulated machine, I crafted the following command.

Terminal

```
→ ricardo homework1 qemu-system-arm
    -machine mcimx6ul-evk
    -kernel babysteps.elf
    -display none
    -monitor stdio
QEMU 11.1.1 monitor - type 'help' for more information
(qemu) stop
(qemu) info registers

CPU#0
R00=00000000 R01=00000000 R02=00000000 R03=00000000
R04=00000000 R05=00000000 R06=00000000 R07=00000000
R08=00000000 R09=00000000 R10=00000000 R11=00000000
R12=00000000 R13=00000000 R14=00000000 R15=00008000
...
FPSCR: 00000000
(qemu) quit
```

I removed part of the output to keep only the important information; I put ... where I cut the info.

The program didn't print any info (because it is not supposed to do so), but I can see that the register `R15=00008000` is moving in the endless loop.

To run this, I had to:

- Install the required packages in my Arch Linux installation using pacman.
  - The required compiler extensions.
  - Utils for binaries.
  - QEMU as the full hardware emulator (`qemu-system-arm`).
- Build a sample assembly `babysteps.s` and then run it in a full hardware emulation of the board `mcimx6ul-evk`.

Now, I investigated (with a little bit of help of my agent) which board in the catalog of emulable boards is the closest to the `USB Armory MKII`.

I found the board `mcimx6ul-evk` as the closest for the following reasons:

- Both use processors from the `NXP i.MX6UL` family.
- Both use an `ARM Cortex-A7 CPU` core.
- The emulated board has peripherals and a memory layout similar to those of an embedded `i.MX6UL` system.

I will continue using this `mcimx6ul-evk` board from now on.

> **Extra Credit** Modify the assembly code to print a string to the serial0 port upon startup.

For this, I modified the file as follows:

babysteps_serial.s

```
1   .equ UART1_UTXD, 0x02020040
2
3   .global _start
4
5   _start:
6       ldr r0, =message
7       ldr r1, =UART1_UTXD
8
9   print_character:
10      ldrb r2, [r0], #1
11      cmp r2, #0
12      beq .
13      str r2, [r1]
14      b print_character
15
16  message:
17      .asciz "Hello from serial0!\r\n"
```

More details about this:

- `.equ UART1_UTXD, 0x02020040` defines the address of the UART1 transmit register, which QEMU connects to the `serial0` output.
- `.global _start` makes the entry symbol globally visible.
- The first two `ldr` instructions store the address of the message in `r0` and the address of the UART transmit register in `r1`.
- `ldrb r2, [r0], #1` reads one byte of the message into `r2` and increments `r0` to point to the next character.
- `cmp r2, #0` and `beq .` detect the null byte at the end of the string and stay at the current instruction forever.
- `str r2, [r1]` writes the character to the UART transmit register, and `b print_character` repeats the process for the next character.
- `.asciz` stores the message with a null byte at the end; `\r\n` are a carriage return and a new line.

Now, building and executing it.

Terminal

```
→ ricardo homework1 arm-none-eabi-as babysteps_serial.s -o babysteps_serial.o
→ ricardo homework1 arm-none-eabi-ld babysteps_serial.o -o babysteps_serial.elf
→ ricardo homework1 qemu-system-arm
    -machine mcimx6ul-evk
    -kernel babysteps_serial.elf
    -display none
    -monitor none
    -serial stdio
Hello from serial0!
```

7 Debugging our environment

Let's install the debugger.

Terminal

```
→ ricardo ~ sudo pacman -S --needed arm-none-eabi-gdb
```

Now, I ran the sample program in one terminal and the debugger in another terminal.

I had to replace the flag `-singlestep` with `-one-insn-per-tb` because the former is not available in the latest version of QEMU `(QEMU 11.1.1)`.

Terminal 1

```
→ ricardo homework1 qemu-arm -one-insn-per-tb -g 1234 babysteps.elf
(Waiting for connection...)
```

Terminal 2

```
→ ricardo homework1 arm-none-eabi-gdb
GNU gdb (GDB) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-pc-linux-gnu --target=arm-none-eabi".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb) file babysteps.elf
Reading symbols from babysteps.elf...
```

```
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
_start () at babysteps.s:5
5           eor r1, r1
(gdb) b babysteps.s:4
Breakpoint 1 at 0x8000: file babysteps.s, line 5.
(gdb) c
Continuing.

Breakpoint 1, _start () at babysteps.s:5
5           eor r1, r1
```

What I did so far is the following step-by-step:

1. I ran `babysteps.elf`, opening the port `1234` for debugging.
2. In the second terminal, I opened the debugger `GDB`.
3. Then, I used `file babysteps.elf` to load the symbols of that file.
4. Then, I connected to the emulated program via port 1234 of the localhost.
5. I set a breakpoint at the beginning of the endless loop; then the program stopped there.
6. Finally, I continued the execution, but the program hit the breakpoint again after going up in the loop.